

Project Proposal: LoopGuard

Team Members: Alice Chen, Bob Smith, Charlie Davis

Repository: github.com/alicechen/loopguard-f26

1. The Product and Target Audience

The Product: LoopGuard is a static analysis linting web application for visual automation workflows. It allows users to upload exported JSON files from automation platforms (like n8n or Zapier) to map out indirect circular dependencies before they are deployed to production.

Target Audience: Operations teams, DevOps engineers, and "citizen developers" who build complex, multi-step automated workflows but lack traditional software engineering tools to analyze cross-application dependencies.

2. The LLM Feature and the Cost of Being Wrong

The Sharp LLM Feature: We will build a focused execution-path parser that reads the JSON export of multiple interconnected workflows. Instead of standard text generation, the LLM will be prompted to act as a dependency resolution engine. It will identify non-obvious trigger-action loops (e.g., Workflow A updates a Salesforce record, which triggers Workflow B, which hits a webhook that retriggers Workflow A) and flag them with a severity score and a plain-English explanation of the collision.

The Cost of Being Wrong:

- **False Positives (Flagging a safe, terminating loop):** The tool blocks the deployment of a legitimate, complex business process, causing the operations team to waste hours hunting for a non-existent architectural flaw.
- **False Negatives (Missing a recursive trigger loop):** The workflows deploy and run infinitely. This rapidly drains thousands of dollars in SaaS API credits, rate-limits critical internal servers, and corrupts production databases with duplicated entries.

3. Scope for the Semester

What we are building (In Scope):

- A data ingestion pipeline that strips bloat from raw n8n/Zapier JSON exports, preserving only the node logic, triggers, and external API actions.
- An automated evaluation pipeline using few-shot prompting to measure the LLM's accuracy against a ground-truth dataset of known looping and safe workflows.
- A lightweight React dashboard that visualizes the detected loop path and explains the vulnerability.

What we are deliberately leaving out (Out of Scope):

4. Execution Plan

Checkpoint	Milestone Focus	Owner	Target Date
Checkpoint 1	Foundation: Repo setup, JSON pre-processing script, and collection of 40 sample workflows (20 safe, 20 looping) for our ground-truth dataset.	Alice Chen	Oct 19, 2026
Checkpoint 2	LLM Prompting & Eval: Establish the JSON-mode prompt schema and build the automated testing script to evaluate LLM accuracy against the ground truth.	Bob Smith	Nov 2, 2026
Checkpoint 3	Backend API: Wrap the parser and OpenAI calls into a functional FastAPI backend; achieve >90% precision on the test dataset.	Charlie Davis	Nov 16, 2026
Checkpoint 4	Frontend MVP: Build the minimal React UI (file upload, loading state, visual presentation of the loop path) and wire it to the API.	Alice Chen	Nov 23, 2026
Checkpoint 5	Refinement: Implement JSON chunking/stripping for massive workflow files that exceed token limits, and improve API latency.	Charlie Davis	Nov 30, 2026
Checkpoint 6	Final Polish: End-to-end bug bashing across the stack, finalized README documentation, and final presentation preparation.	Bob Smith	Dec 7, 2026

5. Top Risks and Mitigations

Risk 1: Token Limits and Context Window Overload

Enterprise workflow JSON exports can be tens of thousands of lines long, containing UI coordinates and aesthetic metadata that will bloat the LLM context window and increase latency/costs.

Mitigation: Before sending data to the LLM, we will write a strict Python pre-processor that strips all positional, visual, and non-functional metadata from the JSON, passing only the node types, trigger conditions, and parameter logic to the model.

Risk 2: Hallucinating Implicit Connections

The LLM might hallucinate connections between two completely isolated workflows just because they happen to interact with the same third-party tool (e.g., Slack), leading to high false-positive rates.

Mitigation: We will heavily utilize few-shot prompting, providing the LLM with explicit negative examples of workflows that share a platform but do **not** loop. We will constantly run regression tests against our 40-workflow dataset whenever the prompt is adjusted.

6. Technology Choices